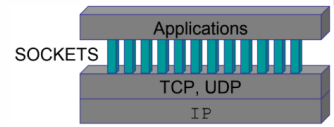
ISIMM | 2025

DÉVELOPPEMENT DES APPLICATIONS RÉPARTIES

Audience: LI3

Enseignant: Dr. Marwa Thabet
Maître assistant @ISIMM

Limites des sockets



- Mécanisme de bas niveau
  - Utilisé pour implémenter des programmes modulaires ou orientés objet.
  - Communication par envoi de messages, contrairement à l'appel de procédure.
- Passage de paramètres
  - Nécessite de réconcilier différents formats de données dans un environnement hétérogène.
  - Codage des données primitives et représentation des structures complexes (tableaux, arbres, objets).
- Gestion dynamique et robustesse
  - Résoudre dynamiquement les références des modules à invoquer, plutôt que de les coder en dur.
  - Détecter et traiter les erreurs de communication (pertes, délais, pannes).

→ **Les sockets nécessitent un système au-dessus d'eux (ex. RPC) pour dépasser leurs limites et offrir une communication plus abstraite.**

Chapitre 2

Les Systèmes RPC et RMI

Plan

1. Interactions dans un Système d'Information Distribué
2. Système RPC
3. RMI - Remote Method Invocation

1. Interactions dans un Système d'Information Distribué

4 problèmes majeurs doivent être résolus:

1) Intégration des primitives de communication dans les langages de programmation
- Comment communiquer ?
- Quel paradigme sous-jacent adopter (procédural, objet, message) ?

2) Réconciliation des représentations machines des données
- Différences d'architecture;
- Représentation des structures de données complexes.

3) Localisation des modules distants
- Déterminer qui connaît l'adresse, et quand elle est résolue.

4) Traitement des erreurs de communication
- Détection, reprise et tolérance aux pannes.

➔ Ces défis sont à la base du passage des sockets vers les systèmes RPC et RMI, qui visent à simplifier la communication entre modules distants.

1. Interactions dans un Système d'Information Distribué

(1) Intégration des primitives de communication dans les langages de programmation

– **Communication implicite:**
- Les primitives réseau sont cachées derrière des instructions normales du langage.
- Le système transforme automatiquement l'appel local en interaction distante.
- Bloquante : l'appelant attend la réponse de l'appelé.
- Avantage : le programmeur garde un modèle de programmation simple.
- Inconvénient : difficile à gérer en cas d'erreurs réseau.

```
result = service.add(5, 10);
```

– **Communication explicite:**
- L'échange est effectué via des primitives réseau explicites.
- Requiert un modèle précisant comment la communication se déroule.
- Souvent non bloquante.
- Avantage : plus de flexibilité dans les schémas de communication.
- Inconvénient : le développeur doit gérer la logique réseau.

```
socket.send(msg);
... // faire autre chose
response = socket.receive();
```


1. Interactions dans un Système d'Information Distribué

(2) Représentation intermédiaire

– Représentation des données

Pour communiquer, les systèmes doivent s'accorder sur la structure et le format des données :
- Ordre des octets
- Types complexes : tableaux, arbres, objets.
- Format des messages.

– Marshalling / Sérialisation
- Marshalling : le processus de transformation d'une représentation de donnés particulière vers une autre représentation plus adéquate à la communication (et le stockage persistant).
- Sérialisation : transformation des données en flux d'octets pour l'envoi réseau.

1. Interactions dans un Système d'Information Distribué

(2) Représentation intermédiaire

– Standardisation
- Deux approches pour résoudre l'hétérogénéité :

→ NxN représentations intermédiaires (inefficace).

→ Une représentation commune standardisée

– Langages de Définition d'Interfaces (IDL)
- Définissent les interfaces et formats de données indépendamment du langage.
- Utilisés par les stubs (client) et skeletons (serveur) pour (dé)sérialiser les appels distants.

1. Interactions dans un Système d'Information Distribué

(3) Localisation des modules (binding)

Pour communiquer, un module doit trouver l'autre :

- Liaison statique : adresse connue à l'avance → rapide mais couplage fort.
- Liaison dynamique : via un registre de service / service de nommage / routeur.
- Avantages : tolérance aux pannes, répartition de charge, découplage.

2. Systèmes RPC

Les sockets sont encore de bas niveau pour plusieurs applications .

Les systèmes **RPC** ont émergé pour:

– Masquer les détails de communication dans un appel de procédure

– Réconcilier des environnements hétérogènes

Elles permettent la communication entre des processus

– Transférer des données

– Invoquer une action dans un autre processus (éventuellement s'exécutant dans une autre machine distante).

2. Systèmes RPC

- **RPC:** Remote Procedure Call – Appelle de procédure à distance
- Le RPC est une abstraction de haut niveau de la communication inter-processus (IPC) à distance. Il permet à un programme d'appeler une procédure située dans un autre processus ou une autre machine comme si elle était locale.
- **Principes clés:**

– Même métaphore et même style de programmation que l'appel de procédure classique.

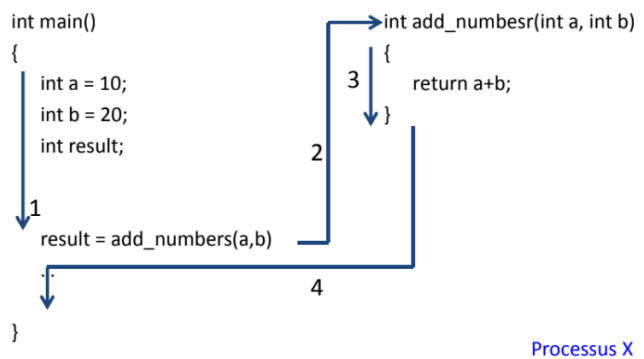
– Masque les détails de communication réseau (envoi, réception, formatage, etc.)

– Appelle une fonction distante de manière transparente.

– Modèle de programmation structuré autour des procédures, et non du passage de messages.

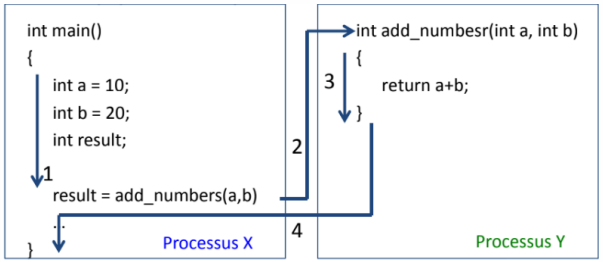
2. Systèmes RPC

Appel de procédure à distance



2. Systèmes RPC

Appel de procédure à distance



• **Flot 1 et 3** sont les mêmes dans les deux cas (local et à distance) **mais** dans deux processus différents thread/processus
• **Flot 2**: l'appel passe de X à Y
• **Flot 4**: le retour passe de Y à X

2. Systèmes RPC

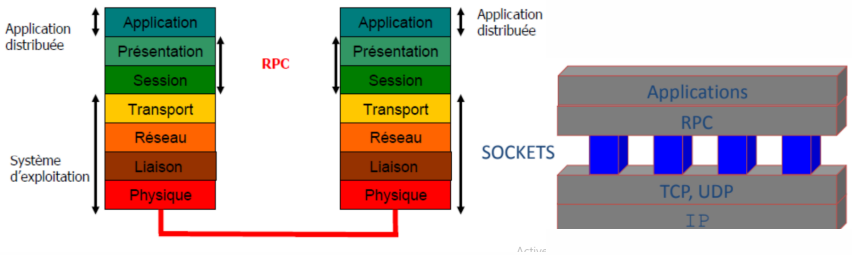
Avantages:

- Cache la complexité du réseau :
  - Sockets, numéros de port
  - Format et ordre des octets
  - Sérialisation / désérialisation des données
- Simplifie la programmation distribuée
- Rend le code plus lisible et maintenable

2. Systèmes RPC

RPC en pratique:

- Le programmeur ne peut pas implémenter toutes les mécanismes nécessaires à chaque fois il développe une application distribuée. Plutôt , ils utilisent/reposent sur un système de RPC (premier exemple d'intergiciel de bas niveau).



2. Systèmes RPC

RPC en pratique:

- Un système RPC:
  - Fournit un langage de définition d'interface (IDL) pour décrire les services.
  - Génère tout le code additionnel nécessaire pour faire un appel de procédure à distance et traiter les différents aspects de communication.
  - Fournit un "binder" en cas il utilise un service de nommage distribué.

2. Systèmes RPC

IDL (Interface Definition Language)

- Tous les systèmes RPC fournissent un langage, IDL, pour décrire les services d'une manière abstraite (indépendamment des langages de programmation).
- Étant donné une spécification IDL, le compilateur d'interface achève un ensemble de tâches pour générer les 'stubs'' et ''skeletons'' dans des IDL définit aussi une représentation intermédiaire des données.
- Un compilateur d'interface est alors utilisé pour générer des ''stubs'' pour les clients et ''skeletons'' des serveurs. Il peut aussi générer des entêtes de procédures que les programmeurs peuvent utiliser et étendre pour compléter l'implémentation des serveurs et des clients.

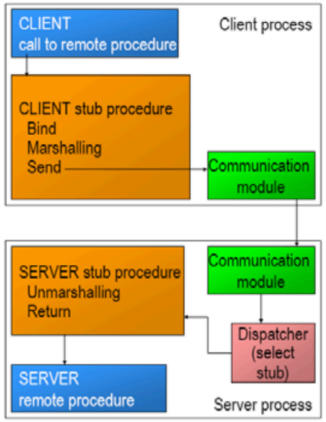
2. Systèmes RPC

IDL (Interface Definition Language)

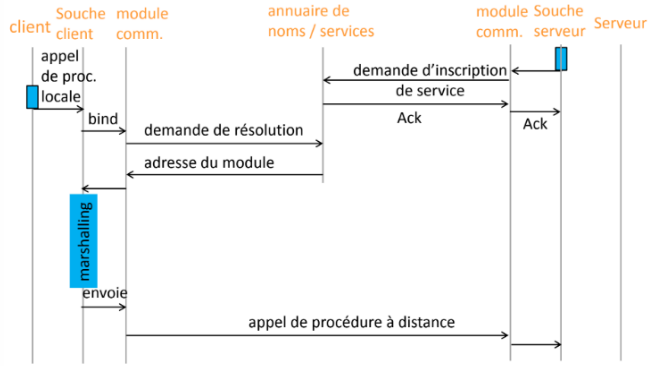
• Étant donné une spécification IDL, le compilateur d'interface achève un ensemble de tâches pour générer les ''stubs'' et ''skeletons'' dans des langages de programmation particuliers (comme JAVA, C) :

1. Génère la procédure de stub du client pour chaque procédure dans l'interface. Le stub va alors être compilé et lié avec le code du client.
2. Génère le skeleton du serveur. Il peut aussi générer une ébauche du code de serveur. Ce code peut être alors complété par le développeur pour implémenter les procédures.
3. Il peut aussi générer des fichiers (d'entête *.h/des classes) des types à importer par les clients et serveurs.

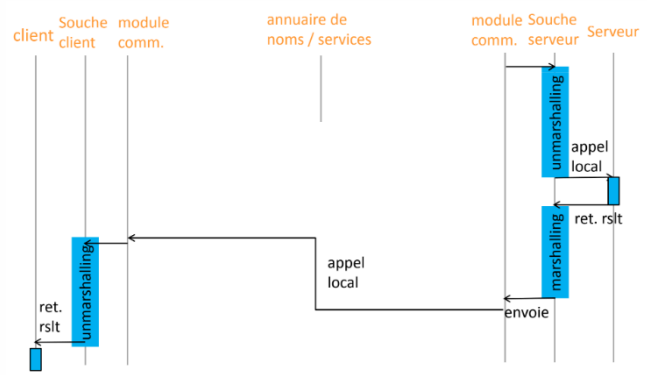
2. Systèmes RPC



2. Systèmes RPC



2. Systèmes RPC



3. RMI - Remote Method Invocation

- Extension orientée objet du concept de RPC, propre à Java.
- Permet d'invoquer des méthodes d'objets distants.
- Démarche pour développement RMI
  1. Créer les interfaces des objets distants
  2. Créer les implémentation des objets distants
  3. Générer les stubs et skeletons
  4. Créer le serveur RMI
  5. Créer le client RMI
  6. Déploiement / Lancement
     - Lancer l'annuaire RMIREGISTRY
     - Lancer le serveur
     - Lancer le client

3. RMI - Remote Method Invocation

(1) Créer les interfaces des objets distants

- La première étape consiste à créer une interface distante qui décrit les méthodes que le client pourra invoquer à distance.
- Pour que ses méthodes soient accessibles par le client, cette interface doit hériter de l'interface Remote.
- Toutes les méthodes utilisables à distance doivent pouvoir lever les exceptions de type RemoteException qui sont spécifiques à l'appel distant.
- Les paramètres sont de type simple objets Sérialisables (implements Serializable) ou Distants (implements Remote).
- Cette interface devra être placée sur les deux machines (serveur et client).

3. RMI - Remote Method Invocation

(1) Créer les interfaces des objets distants

```
import java.rmi.Remote;
import java.rmi.RemoteException;

public interface Hello extends java.rmi.Remote {
String sayHello(String input) throws java.rmi.RemoteException;
}
```

3. RMI - Remote Method Invocation

(2) Créer les implémentation des objets distants

```
import java.rmi.*;
import java rmi server UnicastRemoteObject import
java.rmi.server.UnicastRemoteObject;
public class HelloImpl extends UnicastRemoteObject implements Hello {
private String name;
public HelloImpl() throws RemoteException {super(); }
// Implémentation Implémentation de la méthode méthode distante distante
public String sayHello(String input) throws RemoteException {
return “This is a message from the remote object. Hello from " + input; }}
```


3. RMI - Remote Method Invocation

(3) Générer les stubs et skeletons

• Si l’implémentation est créée, les stubs et skeletons peuvent être générés par l’utilitaire **rmic** en écrivant la commande rmic.

rmic HelloImpl

If you’re using Java 8 or later, you do not need rmic anymore — stubs are generated dynamically.

3. RMI - Remote Method Invocation

(4) Création et enregistrement d’objets distants

• Un serveur crée un service distant en créant d'abord un objet local qui implémente implémente ce service.
• Ensuite, il exporte cet objet vers RMI. Quand l’objet est exporté, RMI crée un service d’écoute qui attend qu’un client se connecte et envoie des requêtes au service.
• Après exportation, le serveur enregistre cet objet dans le registre de RMI sous un nom public qui devient accessible de l’extérieur.
• Le client peut alors consulter le registre distant pour obtenir des références à des objets distants.

3. RMI - Remote Method Invocation

(4) Création et enregistrement d’objets distants

• Les clients trouvent les services distants en utilisant un service d’annuaire d'annuaire activé sur un hôte connu avec un numéro de port connu.
• RMI peut utiliser plusieurs services d'annuaire, y compris **Java Naming** and **Directory Interface (JNDI)**.
• Il inclut lui-même un service simple appelé (rmiregistry).
• Le regitsre est exécuté sur chaque machine qui héberge des objets distants (les serveurs) et accepte les requêtes pour ces services, par défaut sur le port **1099**.

3. RMI - Remote Method Invocation

(4) Création et enregistrement d'objets distants

Notre serveur doit enregistrer auprès du registre RMI l'objet local dont les méthodes seront disponibles à distance.

•Cela se fait grâce à la méthode statique **bind()** ou **rebind()** de la classe **Naming**.

•Cette méthode permet d'associer (enregistrer) l'objet local avec un synonyme dans le registre RMI.

• Le Naming enregistre le nom de l'objet serveur et une souche client (contenant l'@IP et le port de comm.) dans le rmiregistry

• L'objet devient alors accessible par le client.

```
ObjetDistantImpl od = new ObjetDistantImpl();
Naming.rebind("rmi://localhost:1099/NOM_Service",od);
```


3. RMI - Remote Method Invocation

(4) Création et enregistrement d'objets distants

```
import java.rmi.Naming;
public class ServeurRMI {
public static void main(String[] args) {
try {
// Créer l'objet distant
HelloImpl hello=new HelloImpl();
// Publier sa référence dans l'annuaire
Naming.rebind("rmi://localhost:1099/bonjour",hello);
} catch (Exception e) {
e.printStackTrace();
}}}
```


3. RMI - Remote Method Invocation

(5) Créer le client RMI

• Le client peut obtenir une référence à un objet distant par l'utilisation de la méthode statique **lookup()** de la classe Naming.

• La méthode **lookup()** sert au client pour interroger un registre et récupérer un objet distant. Elle retourne une référence à l'objet distant.

• Le **Naming** récupère la souche client de l'objet serveur, la valeur retournée est du type **Remote**. Il est donc nécessaire de caster cet objet en l'interface distante implémentée par l'objet distant.

3. RMI - Remote Method Invocation

(5) Créer le client RMI

```
import java.rmi.Naming;
public class ClientRMI {
public static void main(String[] args) {
try {
Hello stub= (Hello)Naming.lookup("rmi://localhost:1099/bonjour");
System.out.println(stub.sayHello("Galway"));
} catch (Exception e) {
e.printStackTrace();
}
}}
```

3. RMI - Remote Method Invocation

(6) Déploiement Lancement

• Il est maintenant possible de lancer l'application. Cela va nécessiter l'utilisation de trois consoles:

– La première sera utilisée pour activer le registre. Pour cela, vous devez exécuter l'utilitaire rmiregistry : start rmiregistry.

– Dans une deuxième console, exécutez le serveur. Celui-ci va charger l'implémentation en mémoire, enregistrer cette référence dans le registre et attendre une connexion cliente.

– Vous pouvez enfin exécuter le client dans une troisième console.

Même si vous exécutez le client et le serveur sur la même machine, RMI utilisera la pile réseau et TCP/IP pour communiquer entre les JVM.

3. RMI - Remote Method Invocation

JNDI (Java Naming and Directory Interface)

Cette API fournit une interface unique pour utiliser différents services de nommages ou d'annuaires et définit une API standard pour permettre l'accès à ces services. Il existe plusieurs types de service de nommage parmi lesquels :

– DNS (Domain Name System) : service de nommage utilisé sur internet pour permettre la correspondance entre un nom de domaine et une adresse IP.

– LDAP(Lightweight Directory Access Protocol).

– Annuaire NIS (Network Information System) : service de nommage réseau développé par Sun Microsystems.

– COS Naming (Common Object Services) : service de nommage utilisé par Corba pour stocker et obtenir des références sur des objets Corba – RMI Registry : service de nommage utilisé par RMI.

– etc, ...

3. RMI - Remote Method Invocation

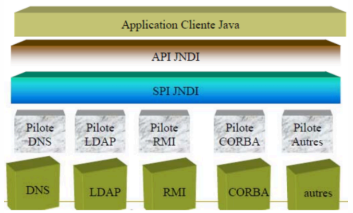
JNDI (Java Naming and Directory Interface)

- Un service de nommage permet d'associer un nom unique à un objet et de faciliter ainsi l'obtention de cet objet.
- Un annuaire est un service de nommage qui possède en plus une représentation hiérarchique des objets qu'il contient et un mécanisme de recherche.
- JNDI propose donc une abstraction pour permettre l'accès à ces différents services de manière standard. Ceci est possible grâce à l'implémentation de pilotes qui mettent en œuvre la partie SPI de l'API JNDI. Cette implémentation se charge d'assurer le dialogue entre l'API et le service utilisé.

3. RMI - Remote Method Invocation

JNDI (Java Naming and Directory Interface)

- L'architecture de JNDI se compose d'une API et d'un Service Provider Interface (SPI). Les applications Java emploient l'API JNDI pour accéder à une variété de services de nommage et d'annuaire. Le SPI permet de relier, de manière transparente, une variété de services de nommage et d'annuaire ; permettant ainsi à l'application Java d'accéder à ces services en utilisant l'API JNDI.



3. RMI - Remote Method Invocation

JNDI (Java Naming and Directory Interface)

- JNDI (Java Naming and Directory Interface) fait partie du JDK standard depuis Java 1.3 (2000). Elle est toujours incluse dans les versions récentes du JDK (Java SE 8, 11, 17, 21, etc.) — aucune installation séparée n'est nécessaire.
- JNDI repose sur les classes de l'API JNDI, fournies dans le package javax.naming.*, et un ou plusieurs fournisseurs (SPI / Service Providers) selon le service de nommage utilisé.
- Les fournisseurs par défaut toujours supportés dans le JDK sont :
  - LDAP (Lightweight Directory Access Protocol)
  - RMI Registry (Remote Method Invocation Registry)
  - DNS (Domain Name System)

3. RMI - Remote Method Invocation

Serveur RMI utilisant JNDI

```
package service;
import java.rmi.Naming;
import java.rmi.registry.LocateRegistry;
public class ServeurRMI {
public static void main(String[] args) {
try {
// Démarrer l'annuaire RMI Registry
LocateRegistry.createRegistry (1099);
// Créer l'Objet distant
BanqueImpl od=new BanqueImpl();
// Créer l'objet InitialContext JNDI en utilisant le fichier jndi.properties
Context ctx=new InitialContext();
// Publier la référence de l'objet distant avec le nom BK
ctx.bind("BK", od);
} catch (Exception e) { e.printStackTrace();}
}
```

```
// Fichier jndi.properties
java.naming.factory.initial=com.sun.jndi.rmi.registry.RegistryContextFactory
java.naming.provider.url=rmi://localhost:1099
```


3. RMI - Remote Method Invocation

Client RMI utilisant JNDI

```
import javax.naming.*; import rmi.IBanque;
public class ClientRMI {
public static void main (String[] args) {
try {
Context ctx=new InitialContext();
ctx.addToEnvironment(Context.INITIAL_CONTEXT_FACTORY,
"com.sun.jndi.rmi.registry.RegistryContextFactory");
ctx.addToEnvironment("java.naming.provider.url", "rmi://localhost:1099");
IBanque B stub = (IBanque B ) ctx.lookup ctx.lookup("BK ");
System.out.println(stub.test("test"));
} catch (Exception e) { e.printStackTrace();}
}}
```

•Dans ce cas, on pas besoin de créer le fichier jndi.properties

MERCI DE VOTRE ATTENTION

Exercice:

Un cabinet dentaire souhaite développer une application client-serveur en Java pour gérer les rendez-vous des clients via une communication par sockets TCP, selon cette spécification :

- Le serveur doit répondre aux requêtes des clients pour consulter les informations des patients en fonction de leurs ID. Si le patient est trouvé, le serveur renvoie ses informations. Sinon, il renvoie un message indiquant que le patient est introuvable. Le serveur doit répondre aux requêtes du client pour réserver un rendez-vous pour un patient en fonction de son ID.
- Le client envoie une demande de prise de rendez-vous avec le nom du patient, la date, l'heure, et le motif (la raison ou au type de service pour lequel le rendez-vous est pris). Si le patient est trouvé, le serveur enregistre le rendez-vous et renvoie une confirmation avec les détails de la réservation. Sinon le serveur renvoie un message indiquant que le patient est introuvable et que le rendez-vous n'a pas pu être pris.

Exercice:

Note: Le serveur stocke une liste de patients avec les informations suivantes : ID, Nom et Age.

Implémenter cette application en permettant au serveur de servir plusieurs clients d'une manière parallèle en utilisant les sockets TCP;

1. Implémenter le programme du serveur (classe Serveur + classe Service) qui doit être installé sur la machine de l'adresse Cabinets.dents.tn sachant que le service de noms doit être activé sur le port numéro 9753.
2. Implémenter le programme du Client